# Appendix A

# ILOC

## A.1 INTRODUCTION

ILOC is a linear assembly code for a simple abstract RISC machine. The ILOC used in this book is a simplified version of the intermediate representation (IR) that was used in the Massively Scalar Compiler Project (MSCP) at Rice University. For example, ILOC as defined here assumes just two base data types, an integer of unspecified length, and a single character. In the MSCP compiler, the IR supported a much broader set of data types.

The ILOC abstract machine has an unlimited number of registers. It has register-to-register operations; load and store operations; comparisons; and branches. It supports four memory address modes: *direct*, *address-immediate*, *address-offset*, and *immediate*. Source operands are read at the start of the cycle in which the operation issues. Result operands are defined at the end of the cycle in which the operation completes.

Other than its instruction set, the details of the machine are left unspecified. Most of the examples in this book assume a simple machine, with a single functional unit that executes ILOC operations in their order of appearance. When other configurations are used, we discuss them explicitly.

An ILOC program consists of a sequential list of instructions, as follows:

*IlocProgram* → *InstructionList*

*InstructionList* → *Instruction*
| *Instruction InstructionList*

An instruction consists of one or more operations, with an optional label.

*Instruction* → *OperationList*
| `label :` *OperationList*

A label is an alphanumeric string, ( [*A*–*Z*] | [*a*–*z*] ) ( [*A*–*Z*] | [*a*–*z*] | [0–9] )*. A colon follows the label. If the code needs multiple labels at some point, we insert labeled `nop` instructions at the appropriate location.

An *OperationList* consists of either a single *Operation*, or a list of *Operation*s surrounded by square brackets and separated by semicolons.

```
OperationList  →  Operation
               |  [ ListOfOps ]
ListOfOps      →  Operation
               |  Operation ; ListOfOps
Operation      →  NormalOp
               |  ControlFlowOp
```

An ILOC operation corresponds to a single machine-level operation. An *Operation* is either a *NormalOp* that performs computation or data movement, or it is a *ControlFlowOp* used to change the flow of control in the program.

*NormalOp*s consist of an opcode, a list of comma-separated source operands, and a list of comma-separated result operands. The sources are separated from the results by the symbol ⇒, pronounced "into."

```
NormalOp       →  Opcode OperandList ⇒ OperandList
OperandList    →  Operand
               |  Operand , OperandList
Operand        →  register
               |  number
               |  label
```

Unfortunately, as in a real assembly language, the relationship between an opcode and the form of its operands is not systematic. The easiest way to specify the form of the operands for each opcode is in a tabular form. The tables in Section A.6 show the full set of opcodes and their operands for each of the ILOC operations that appear in the book.

*Operand*s may be one of three types: `register`, `number`, or `label`. The type of each operand is determined by the opcode and the position of the operand in the operation. In the examples, we use both numerical ($r_{10}$) and symbolic ($r_i$) names for registers. Numbers are simple positive integers. Labels are, as defined earlier, alphanumeric strings, ( [*A*–*Z*] | [*a*–*z*] ) ( [*A*–*Z*] | [*a*–*z*] | [0–9] )*.

While the definition of a label allows `r10` and `ri` as labels, we avoid labels that could have other interpretations.

Arithmetic operations have one result; some of the `store` operations have two results, as do all of the branches.

As an example, `storeAI`, the *address-immediate* store operation, has one source operand (a `register`) and two result operands (a `register` and a `number`). The operation `storeAI` $r_i \Rightarrow r_j, 4$ adds 4 to the value in $r_j$ to form an address and stores the value $r_i$ into the memory location at that address. It performs the action: MEMORY ($r_j + 4$) ← CONTENTS($r_i$).

## A.2 NAMING CONVENTIONS

The ILOC code used in examples throughout the text follows a simple set of naming conventions.

1. Memory offsets for variables are represented symbolically by prefixing the variable name with the `@` character.
2. The user can assume an unlimited supply of registers. These are named with simple integers, as in $r_{1776}$, or with symbolic names, as in $r_i$.
3. The register $r_{arp}$ is reserved for a pointer to the current activation record. Thus, the operation `loadAI` $r_{arp}$, `@x` $\Rightarrow$ $r_1$ loads an integer from offset `@x` in the current activation record into $r_1$.

ILOC comments begin with the string // and continue until the end of a line. We assume that these are stripped out by the scanner.

## A.3 COMPUTATIONAL OPERATIONS

The first major category of operations in ILOC is the set of basic computational operations. ILOC provides these operations in both a three-address, register-to-register format and a two-register format that has an immediate constant as its second source operand.

**Register-to-Register Operations**

| Opcode | Sources | Results | Meaning |
|---|---|---|---|
| add | $r_1, r_2$ | $r_3$ | $r_1 + r_2 \Rightarrow r_3$ |
| sub | $r_1, r_2$ | $r_3$ | $r_1 - r_2 \Rightarrow r_3$ |
| mult | $r_1, r_2$ | $r_3$ | $r_1 \times r_2 \Rightarrow r_3$ |
| div | $r_1, r_2$ | $r_3$ | $r_1 \div r_2 \Rightarrow r_3$ |
| lshift | $r_1, r_2$ | $r_3$ | $r_1 \ll r_2 \Rightarrow r_3$ |
| rshift | $r_1, r_2$ | $r_3$ | $r_1 \gg r_2 \Rightarrow r_3$ |

The first four ocodes implement standard arithmetic operations. The latter two opcodes implement logical shift operations.

A real ILOC processor or a full-fledged compiler would need more than one arithmetic data type. This would lead to typed opcodes or to polymorphic opcodes. The MSCP compiler had distinct arithmetic operations for various lengths of integer and floating-point numbers, as well as for pointers.

The next group of opcodes specify register-immediate operations. Each of them takes as input a register and an immediate constant value. The noncom-

mutative operations have two distinct forms to allow the constant as either the first or second value in the operation. For example, `rsubI` subtracts the value in $r_1$ from the immediate constant $c_2$.

| Register-Immediate Operations | | | |
|---|---|---|---|
| **Opcode** | **Sources** | **Results** | **Meaning** |
| addI | $r_1, c_2$ | $r_3$ | $r_1 + c_2 \Rightarrow r_3$ |
| subI | $r_1, c_2$ | $r_3$ | $r_1 - c_2 \Rightarrow r_3$ |
| rsubI | $r_1, c_2$ | $r_3$ | $c_2 - r_1 \Rightarrow r_3$ |
| multI | $r_1, c_2$ | $r_3$ | $r_1 \times c_2 \Rightarrow r_3$ |
| divI | $r_1, c_2$ | $r_3$ | $r_1 \div c_2 \Rightarrow r_3$ |
| rdivI | $r_1, c_2$ | $r_3$ | $c_2 \div r_1 \Rightarrow r_3$ |
| lshiftI | $r_1, c_2$ | $r_3$ | $r_1 \ll c_2 \Rightarrow r_3$ |
| rshiftI | $r_1, c_2$ | $r_3$ | $r_1 \gg c_2 \Rightarrow r_3$ |

A register-immediate operation is useful for three distinct reasons.

- It lets the compiler or the assembly-level programmer use a small constant directly in the operation. Thus, it decreases demand for registers. It may also eliminate an operation to load the constant into a register.
- It "reads" the constant from the instruction stream and, thus, from the instruction cache. Most processors have separate caches and data paths for instructions and for data; thus, the immediate operation uses fewer resources on the data-side of the memory interfaces.
- The immediate form of the operation records the value of the constant in a visible and easily accessible way, which ensures that optimization and code generation can see and use the value.

## A.4 DATA MOVEMENT OPERATIONS

The second major category of operations in ILOC allow the compiler or the assembly-level programmer to specify data movement. We include two specialized operations in this section: a conditional move operation and a representation for a $\phi$-function (see Section 4.6.2).

### Memory Operations

ILOC provides a set of operations to move values between memory and registers: `load` and `store` operations. The examples in the book limit themselves to integer and character data; a complete version of ILOC would need `load` and `store` operations for a much broader variety of base types.

To produce efficient code, a compiler's back end must make effective use of the address modes provided on `load` and `store` operations. ILOC provides a variety of address modes in both the `load` and `store` operations.

Most processors use a separate adder to perform the arithmetic in an address computation. Thus, moving an addition into the address mode frees up an issue slot on another functional unit.

**Load Operations**

| Opcode | Sources | Results | Meaning |
|---|---|---|---|
| load | $r_1$ | $r_2$ | MEMORY($r_1$) $\Rightarrow$ $r_2$ |
| loadAI | $r_1$, $c_2$ | $r_3$ | MEMORY($r_1 + c_2$) $\Rightarrow$ $r_3$ |
| loadAO | $r_1$, $r_2$ | $r_3$ | MEMORY($r_1 + r_2$) $\Rightarrow$ $r_3$ |
| cload | $r_1$ | $r_2$ | character load |
| cloadAI | $r_1$, $c_2$ | $r_3$ | character loadAI |
| cloadAO | $r_1$, $r_2$ | $r_3$ | character loadAO |
| loadI | $c_1$ | $r_2$ | $c_1 \Rightarrow r_2$ |

The load operations differ in the address modes that they support.

- The *direct loads*, `load` and `cload`, use the value in $r_1$ as an address.
- The *address-immediate loads*, `loadAI` and `cloadAI`, add the immediate constant and the value in the source register to form the address.
- The *address-offset loads*, `loadAO` and `cloadAO`, add the contents of the two source registers to form the address.
- The *immediate load*, `loadI`, moves the constant into the target register.

A complete, ILOC-like IR with multiple base types for values should have an immediate load for each distinct base type.

The store operations support the same address modes as the load operations.

**Store Operations**

| Opcode | Sources | Results | Meaning |
|---|---|---|---|
| store | $r_1$ | $r_2$ | $r_1 \Rightarrow$ MEMORY($r_2$) |
| storeAI | $r_1$ | $r_2$, $c_3$ | $r_1 \Rightarrow$ MEMORY($r_2 + c_3$) |
| storeAO | $r_1$ | $r_2$, $r_3$ | $r_1 \Rightarrow$ MEMORY($r_2 + r_3$) |
| cstore | $r_1$ | $r_2$ | character store |
| cstoreAI | $r_1$ | $r_2$, $c_3$ | character storeAI |
| cstoreAO | $r_1$ | $r_2$, $r_3$ | character storeAO |

ILOC has no store immediate operation.

### Register-to-Register Copy Operations

To move values directly between registers ILOC includes a set of register-to-register copy operations.

| Copy Operations | | | |
|---|---|---|---|
| **Opcode** | **Sources** | **Results** | **Meaning** |
| i2i | $r_1$ | $r_2$ | $r_1 \Rightarrow r_2$ for integer values |
| c2c | $r_1$ | $r_2$ | $r_1 \Rightarrow r_2$ for character values |
| c2i | $r_1$ | $r_2$ | convert character to integer |
| i2c | $r_1$ | $r_2$ | convert integer to character |

i2i and c2c simply copy a value from one register to another. By contrast, c2i and i2c convert the value to another type as part of the copy operation.

### Conditional Move Operation

Many ISAs provide a conditional move operation that selects a value from one of two source registers based on a Boolean value in a third register. To enable examples that discuss this kind of operation, ILOC includes four-operand conditional move operations for integers and characters.

Architects include conditional move operations in an ISA to let the compiler avoid branches on particularly simple conditional constructs.

| Conditional Move Operations | | | |
|---|---|---|---|
| **Opcode** | **Sources** | **Results** | **Meaning** |
| c_i2i | $r_b, r_1, r_2$ | $r_3$ | $r_1 \Rightarrow r_3$, *if* $r_b$ = true<br>$r_2 \Rightarrow r_3$, *otherwise* |
| c_c2c | $r_b, r_1, r_2$ | $r_3$ | $r_1 \Rightarrow r_3$, *if* $r_b$ = true<br>$r_2 \Rightarrow r_3$, *otherwise* |

### Representing $\phi$ Functions

When a compiler builds SSA form, it must represent $\phi$-functions. In ILOC, the natural way to write a $\phi$-function is as an ILOC operation:

```
phi r_i, r_j, r_k ⇒ r_m
```

for the $\phi$-function $r_m \leftarrow \phi(r_i, r_j, r_k)$. Because of the nature of SSA form, the phi operation may take an arbitrary number of sources. It always defines

a single target register. Representing $\phi$-functions in this way poses some interesting engineering issues in the implementation of the IR.

## A.5 CONTROL-FLOW OPERATIONS

Control-flow operations allow the ILOC program to sequence the execution of labeled blocks of operations. ILOC includes jumps, comparisons, and conditional branches. To facilitate the discussion in Section 7.4.2, ILOC also includes a syntax to represent predicated operations.

While control-flow operations still fit the three-address form of most ILOC operations, they vary in the meanings of their various operands. As a visual cue, the operations that involve a transfer of control (or an assignment to the program counter) use a single arrow, $\rightarrow$, to separate sources from targets.

*ControlFlowOp* → `jumpI` → `label`
| `jump` → `register`
| *Branch* `register` → `label` , `label`
| *Compare* `register`, `register` ⇒ `register`

ILOC provides two styles of *Branch* and *Compare* operations so that Section 7.4.2 can discuss translation of control-flow constructs for each of them.

### Jumps

ILOC has both *jump immediate* and *jump to register* operations. Most examples use an immediate jump because it exposes the destination label.

| Opcode | Sources | Results | Meaning |
|---|---|---|---|
| `jumpI` | -- | $l_1$ | $l_1 \rightarrow$ PC |
| `jump` | -- | $r_1$ | $r_1 \rightarrow$ PC |

The `jump` operation is ambiguous because the target address is in a register. The compiler may be unable to deduce the correct set of possible target labels for a `jump`. This effect complicates CFG construction and can degrade the quality of information that data-flow analysis can derive. For these reasons, we prefer `jumpI` over `jump` whenever possible and practical.

Sometimes the compiler must generate a `jump` rather than a `jumpI`. For these situations, ILOC includes a pseudooperation that lets the compiler record the set of possible labels for a `jump` operation. The `tbl` pseudooperation has two arguments, a register and an immediate label.

| Opcode | Sources | Results | Meaning |
|---|---|---|---|
| tbl | $r_1$, $l_2$ | -- | $r_1$ *might hold* $l_2$ |

A `tbl` operation can occur only after a `jump`. The sequence

```
jump          → ri
tbl   ri, L01
tbl   ri, L03
tbl   ri, L05
tbl   ri, L08
```

asserts that the jump targets one of `L01`, `L03`, `L05`, or `L08`, and no other label.

### Comparisons and Conditional Branches

All branches in ILOC have a label for the true path and the false path. ILOC does not have the notion of a "fall-through" path.

ILOC supports two models for branches. The standard ILOC model, shown in Fig. A.1(a), uses a simple conditional branch, `cbr`, and a set of six Boolean-valued comparison operations. `cbr` tests the Boolean value produced by a comparison and branches to the appropriate immediate label.

The second branch model, shown in panel (b), has one comparison operation and a set of six conditional branches. It models the situation on an ISA where comparison sets a "condition code" (see Section 7.4.2). This model pushes the choice of relational operator into the branch operation.

Actual ISAs vary in the number of condition code registers they support.

In the examples, we designate the target of `comp` as a condition-code register by writing it as $cc_i$. The `comp` operation writes a value drawn from the set {`LT`, `LE`, `EQ`, `GE`, `GT`, `NE`,} into $cc_i$. The conditional branch has a variant for each of these six values.

### Predicated Execution

Predication is another feature introduced to let the compiler or assembly programmer avoid a conditional branch. A predicated operation takes a Boolean value that determines whether or not the operation's result takes effect. In ILOC, a predicated operation is written as follows:

```
(rp) ? add  r1, r2  ⇒  r3
```

This notation indicates that $r_3$ receives the sum of the values of $r_1$ and $r_2$ if $r_p$ contains the value `true`. If $r_p$ does not contain `true`, then $r_3$ is unchanged.

In principle, any ILOC operation can be predicated. In practice, predication does not make sense for some operations, such as a `phi` or a `jump`. (A predicated `jump` is just a `cbr` whose false path is the next operation.)

| Opcode | Sources | Results | Meaning | |
|---|---|---|---|---|
| cmp_LT | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 < r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cmp_LE | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 \leq r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cmp_EQ | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 = r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cmp_GE | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 \geq r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cmp_GT | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 > r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cmp_NE | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 \neq r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cbr | $r_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $r_1 =$ true |
| | | | $l_3 \rightarrow$ PC | *otherwise* |

(a) The Standard Branch Syntax

| Opcode | Sources | Results | Meaning | |
|---|---|---|---|---|
| comp | $r_1, r_2$ | $cc_3$ | *sets* $cc_3$ | |
| cbr_LT | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3 =$ LT |
| | | | $l_3 \rightarrow$ PC | *otherwise* |
| cbr_LE | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3 =$ LE |
| | | | $l_3 \rightarrow$ PC | *otherwise* |
| cbr_EQ | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3 =$ EQ |
| | | | $l_3 \rightarrow$ PC | *otherwise* |
| cbr_GE | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3 =$ GE |
| | | | $l_3 \rightarrow$ PC | *otherwise* |
| cbr_GT | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3 =$ GT |
| | | | $l_3 \rightarrow$ PC | *otherwise* |
| cbr_NE | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3 =$ NE |
| | | | $l_3 \rightarrow$ PC | *otherwise* |

(b) Branch Syntax to Model an ISA with Condition Codes

■ **FIGURE A.1** Conditional Branch Syntax.

## A.6 OPCODE SUMMARY TABLES

The following pages contain summary tables that list all of the ILOC operations and their meanings. They are provided for reference.

| Arithmetic Operations | | | |
|---|---|---|---|
| **Opcode** | **Sources** | **Results** | **Meaning** |
| nop | *none* | *none* | *single cycle, side-effect free operation* |
| add | $r_1, r_2$ | $r_3$ | $r_1 + r_2 \Rightarrow r_3$ |
| sub | $r_1, r_2$ | $r_3$ | $r_1 - r_2 \Rightarrow r_3$ |
| mult | $r_1, r_2$ | $r_3$ | $r_1 \times r_2 \Rightarrow r_3$ |
| div | $r_1, r_2$ | $r_3$ | $r_1 \div r_2 \Rightarrow r_3$ |
| addI | $r_1, c_2$ | $r_3$ | $r_1 + c_2 \Rightarrow r_3$ |
| subI | $r_1, c_2$ | $r_3$ | $r_1 - c_2 \Rightarrow r_3$ |
| rsubI | $r_1, c_2$ | $r_3$ | $c_2 - r_1 \Rightarrow r_3$ |
| multI | $r_1, c_2$ | $r_3$ | $r_1 \times c_2 \Rightarrow r_3$ |
| divI | $r_1, c_2$ | $r_3$ | $r_1 \div c_2 \Rightarrow r_3$ |
| rdivI | $r_1, c_2$ | $r_3$ | $c_2 \div r_1 \Rightarrow r_3$ |
| lshift | $r_1, r_2$ | $r_3$ | $r_1 \ll r_2 \Rightarrow r_3$ |
| lshiftI | $r_1, c_2$ | $r_3$ | $r_1 \ll c_2 \Rightarrow r_3$ |
| rshift | $r_1, r_2$ | $r_3$ | $r_1 \gg r_2 \Rightarrow r_3$ |
| rshiftI | $r_1, c_2$ | $r_3$ | $r_1 \gg c_2 \Rightarrow r_3$ |
| and | $r_1, r_2$ | $r_3$ | $r_1 \wedge r_2 \Rightarrow r_3$ |
| andI | $r_1, c_2$ | $r_3$ | $r_1 \wedge c_2 \Rightarrow r_3$ |
| or | $r_1, r_2$ | $r_3$ | $r_1 \vee r_2 \Rightarrow r_3$ |
| orI | $r_1, c_2$ | $r_3$ | $r_1 \vee c_2 \Rightarrow r_3$ |
| xor | $r_1, r_2$ | $r_3$ | $r_1$ *xor* $r_2 \Rightarrow r_3$ |
| xorI | $r_1, c_2$ | $r_3$ | $r_1$ *xor* $c_2 \Rightarrow r_3$ |

| Data Movement Operations | | | |
|---|---|---|---|
| **Opcode** | **Sources** | **Results** | **Meaning** |
| loadI | $c_1$ | $r_2$ | $c_1 \Rightarrow r_2$ |
| load | $r_1$ | $r_2$ | MEMORY($r_1$) $\Rightarrow$ $r_2$ |
| loadAI | $r_1$, $c_2$ | $r_3$ | MEMORY($r_1 + c_2$) $\Rightarrow$ $r_3$ |
| loadAO | $r_1$, $r_2$ | $r_3$ | MEMORY($r_1 + r_2$) $\Rightarrow$ $r_3$ |
| cload | $r_1$ | $r_2$ | *character* load |
| cloadAI | $r_1$, $c_2$ | $r_3$ | *character* loadAI |
| cloadAO | $r_1$, $r_2$ | $r_3$ | *character* loadAO |
| store | $r_1$ | $r_2$ | $r_1 \Rightarrow$ MEMORY($r_2$) |
| storeAI | $r_1$ | $r_2$, $c_3$ | $r_1 \Rightarrow$ MEMORY($r_2 + c_3$) |
| storeAO | $r_1$ | $r_2$, $r_3$ | $r_1 \Rightarrow$ MEMORY($r_2 + r_3$) |
| cstore | $r_1$ | $r_2$ | *character* store |
| cstoreAI | $r_1$ | $r_2$, $c_3$ | *character* storeAI |
| cstoreAO | $r_1$ | $r_2$, $r_3$ | *character* storeAO |
| i2i | $r_1$ | $r_2$ | $r_1 \Rightarrow r_2$ *for integers* |
| c2c | $r_1$ | $r_2$ | $r_1 \Rightarrow r_2$ *for characters* |
| c2i | $r_1$ | $r_2$ | *convert character to integer* |
| i2c | $r_1$ | $r_2$ | *convert integer to character* |
| phi | $r_1$, $r_2$, $r_3$ | $r_4$ | $\phi(r_1, r_2, r_3) \Rightarrow r_4$ *with arbitrary number of sources* |
| c_i2i | $r_b$, $r_1$, $r_2$ | $r_3$ | $r_1 \Rightarrow r_3$, *if* $r_b$ = true<br>$r_2 \Rightarrow r_3$, *otherwise* |
| c_c2c | $r_b$, $r_1$, $r_2$ | $r_3$ | $r_1 \Rightarrow r_3$, *if* $r_b$ = true<br>$r_2 \Rightarrow r_3$, *otherwise* |

| Jumps | | | |
|---|---|---|---|
| **Opcode** | **Sources** | **Results** | **Meaning** |
| jump | — | $r_1$ | $r_1 \rightarrow$ PC |
| jumpI | — | $l_1$ | $l_1 \rightarrow$ PC |
| tbl | $r_1$, $l_2$ | — | $r_1$ *might hold* $l_2$ |

| Standard Conditional Branch Syntax | | | | |
|---|---|---|---|---|
| **Opcode** | **Sources** | **Results** | **Meaning** | |
| cmp_LT | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 < r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cmp_LE | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 \leq r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cmp_EQ | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 = r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cmp_GE | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 \geq r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cmp_GT | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 > r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cmp_NE | $r_1, r_2$ | $r_3$ | true $\Rightarrow r_3$ | *if* $r_1 \neq r_2$ |
| | | | false $\Rightarrow r_3$ | *otherwise* |
| cbr | $r_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $r_1$ = true |
| | | | $l_3 \rightarrow$ PC | *otherwise* |

| Alternate Conditional Branch Syntax | | | | |
|---|---|---|---|---|
| **Opcode** | **Sources** | **Results** | **Meaning** | |
| comp | $r_1, r_2$ | $cc_3$ | *sets* $cc_3$ *to one of* {LT, LE, EQ, GE, GT, NE} | |
| cbr_LT | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3$ = LT |
| | | | $l_3 \rightarrow$ PC | *otherwise* |
| cbr_LE | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3$ = LE |
| | | | $l_3 \rightarrow$ PC | *otherwise* |
| cbr_EQ | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3$ = EQ |
| | | | $l_3 \rightarrow$ PC | *otherwise* |
| cbr_GE | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3$ = GE |
| | | | $l_3 \rightarrow$ PC | *otherwise* |
| cbr_GT | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3$ = GT |
| | | | $l_3 \rightarrow$ PC | *otherwise* |
| cbr_NE | $cc_1$ | $l_2, l_3$ | $l_2 \rightarrow$ PC | *if* $cc_3$ = NE |
| | | | $l_3 \rightarrow$ PC | *otherwise* |